



SWI-Prolog – trace, read, write

trace/0 — śledzenie wykonania programu

trace/0 służy do debugowania programu. Po jego włączeniu Prolog przechodzi w tryb śledzenia i pokazuje kolejne etapy rozwiązywania celu. Dokumentacja opisuje trace mode jako główny, tekstowy debugger Prologa, który śledzi przejścia między stanami rozwiązywania predykatów reprezentowanymi przez tzw. porty modelu Byrda. Tryb ten można wyłączyć przez notrace/0.

Przykład

```
ojciec(jan, piotr).  
matka(anna, piotr).  
  
rodzic(X, Y) :- ojciec(X, Y).  
rodzic(X, Y) :- matka(X, Y).
```

Zapytanie:

```
?- trace.  
true.
```

```
[trace] ?- rodzic(X, piotr).
```

W czasie wykonania Prolog będzie pokazywał kolejne kroki, na przykład:

- Call — wywołanie celu,
- Exit — cel zakończył się sukcesem,
- Fail — cel zakończył się niepowodzeniem,
- Redo — Prolog wraca, aby szukać kolejnego rozwiązania.

To właśnie te porty tworzą podstawowy obraz działania programu podczas debugowania.

```
?- trace.  
?- rodzic(X, piotr).
```

to Prolog będzie pokazywał, czy najpierw próbuje dopasować regułę z ojciec/2, potem z matka/2, a przy kolejnych rozwiązaniach przechodzi przez Redo. Dzięki temu można zobaczyć:

- w jakiej kolejności sprawdzane są klauzule,
- gdzie następuje porażka,
- dlaczego pojawiają się kolejne odpowiedzi.

write/1 — wypisywanie termów

write/1 zapisuje term na bieżące wyjście i używa nawiasów oraz operatorów tam, gdzie jest to właściwe. Predykaty bez argumentu strumienia, takie jak write/1 i read/1, korzystają z current_output i current_input.

Przykłady

```
?- write(anna).
anna
true.
?- write(rodzic(jan, piotr)).
rodzic(jan, piotr)
true.
```

W obu przypadkach write/1 wypisuje term w postaci czytelnej dla użytkownika.

Przykład w predykanie

```
pokaz_komunikat :-
    write('Witaj studentcie').
```

Zapytanie:

```
?- pokaz_komunikat.
Witaj studentcie
true.
```

write/1 **nie dodaje automatycznie nowej linii**. Jeżeli chcesz przejść do nowej linii, zwykle dodaje się nl albo zamiast write/1 używa się wygodniejszych predykatów, np. writeln/1 lub format/2.

Przykład łączenia write/1 z nl

```
pokaz_osobe(Imie) :-
    write('Osoba: '),
    write(Imie),
    nl.
```

Zapytanie:

```
?- pokaz_osobe(ewa).
Osoba: ewa
true.
```

write/1 jest wygodny, gdy:

- chcesz szybko podejrzeć wynik działania programu,
- budujesz prosty komunikat diagnostyczny,
- chcesz wypisać term bez bardziej rozbudowanego formatowania.

read/1 — odczyt termu Prologowego

read/1 odczytuje następny term Prologowy z bieżącego wejścia i unifikuje go ze zmienną. Po dojściu do końca strumienia zwraca atom end_of_file. Dokumentacja wyraźnie zaznacza też, że term może obejmować wiele linii, ale musi kończyć się kropką oraz znakiem odstępu po niej. read/1 jest równoważne read_term/2 z pustą listą opcji.

Przykład

```
?- read(X).
 |: anna.
X = anna.
```

Użytkownik wpisuje anna. i zatwierdza Enterem. Prolog odczytuje to jako atom anna.

Przykład z termem złożonym

```
?- read(T).  
|: rodzic(jan, piotr).  
T = rodzic(jan, piotr).
```

Tutaj odczytany zostaje pełny term złożony. To bardzo ważne: read/1 nie czyta „dowolnego tekstu”, lecz składnię Prologa.

Przykład w programie

```
wczytaj_i_pokaz :-  
    write('Podaj term: '),  
    read(X),  
    write('Wczytano: '),  
    write(X),  
    nl.
```

Możliwe wykonanie:

```
?- wczytaj_i_pokaz.  
Podaj term: |: student(anna).  
Wczytano: student(anna)  
true.
```

Bardzo ważne zastrzeżenie

Dokumentacja SWI-Prolog podkreśla, że read/1 często nie jest tym, czego naprawdę potrzeba do zwykłego wejścia od użytkownika. Ten predykat służy do czytania termów Prologowych, a nie zwykłego tekstu wpisywanego przez użytkownika. Jest szczególnie użyteczny przy zapisie i odczycie danych Prologowych, np. z plików lub przez komunikację między procesami.

Krótki przykład razem

```
echo_term :-  
    read(X),  
    write(X),  
    nl.
```

Uruchomienie:

```
?- echo_term.  
|: [a,b,c].  
[a,b,c]  
true.
```

To klasyczny mini-przykład pokazujący, że Prolog odczytuje term i od razu go wypisuje. Działa to poprawnie właśnie dlatego, że [a,b,c]. jest legalnym termem Prologowym.

Kiedy stosować te predykaty

- trace/0 stosuj wtedy, gdy program działa niepoprawnie i chcesz zobaczyć krok po kroku, jak Prolog próbuje rozwiązać cel. Szczególnie pomaga przy regułach, rekurencji i wielu możliwych odpowiedziach.
- write/1 stosuj do prostego wypisywania wyników i komunikatów diagnostycznych. Dla bardziej eleganckiego formatowania zwykle wygodniejszy bywa format/2.
- read/1 stosuj wtedy, gdy naprawdę chcesz odczytać term Prologowy. Do zwykłego tekstu użytkownika zwykle lepiej użyć innych narzędzi wejścia.